

Building a Quant Data Pipeline for Nifty 50: A Developer's Log

Shadow Oak Capitals – Summer Sprint 2026

Ryan Kaushal

July 5, 2026

Abstract

I built a Python data pipeline to compute financial statistics for the Nifty 50 index. Starting from zero coding experience, I downloaded five years of daily price data, cleaned it, and computed annualised returns, volatility, Sharpe ratio, and maximum drawdown. The entire pipeline is reproducible, open-source, and manually verified. This paper documents my process, the mistakes I made, and the numbers I got.

1. Introduction

I started this sprint with zero coding experience. On Day 1, I didn't know what a variable was. By Day 10, I had a working data pipeline that downloaded Nifty 50 data and calculated Sharpe ratios. This paper is my honest log of that journey — the errors, the fixes, and the final results.

Why did I build this toolkit? Because I wanted to trade systematically, not on gut feel. The first brick in that wall is a reliable data engine. I needed to know that the numbers I see are correct — that I can trust them before I put real money on the line.

This paper covers:

- The data source and why I chose it.
 - The cleaning steps and why they matter.
 - The mathematical formulas I used.
 - The implementation details.
 - The final results and manual verification.
-

2. Data

2.1. Source

I used **yfinance** (Yahoo Finance) to download historical price data for the Nifty 50 index (ticker **^NSEI**). The download covered **5 years** — from July 2021 to July 2026 — giving me 1,233 trading days.

The data includes:

- Open
- High
- Low
- Close

- Adjusted Close (used for all calculations)
- Volume

2.2. Data Quality

The first download failed. The column names were nested (a multi-index), and Python couldn't find 'Adj Close'. I fixed this by adding `auto_adjust=False` and flattening the column structure.

I also removed the first row of data because the return calculation uses a shifted price, which produces a NaN value on the first day. I kept all other rows — no outlier removal.

2.3. Why Five Years?

Five years gives a sample size of ~1,250 observations. This is large enough for statistical significance but recent enough to reflect current market dynamics.

3. Methodology

3.1. Simple Returns vs. Log Returns

I computed two types of daily returns:

- **Simple return:**

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

This is the percentage change in price.

- **Log return:**

$$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right)$$

Log returns are additive over time, which makes them mathematically convenient for multi-period analysis.

For short horizons (daily), the two measures are nearly identical.

3.2. Annualisation

I annualised returns and volatility using 252 trading days — the industry standard for equities:

$$R_{\text{ann}} = (1 + \bar{r})^{252} - 1$$

$$\sigma_{\text{ann}} = \sigma_{\text{daily}} \times \sqrt{252}$$

3.3. Sharpe Ratio

I computed the Sharpe ratio with a zero risk-free rate:

$$\text{Sharpe} = \frac{R_{\text{ann}}}{\sigma_{\text{ann}}}$$

A higher Sharpe ratio indicates better risk-adjusted performance.

3.4. Maximum Drawdown

Maximum drawdown measures the worst peak-to-trough decline:

$$\text{Max DD} = \min \left(\frac{P_t - \max_{s \leq t} P_s}{\max_{s \leq t} P_s} \right)$$

This is a critical risk metric for any investor.

3.5. Code Implementation

I wrote three Python scripts:

- `downloader.py` — fetches data from yfinance and saves it as CSV.
- `analyzer.py` — loads the CSV, computes metrics, and saves results.
- `visualize.py` — generates a price chart.

The code uses `pandas`, `numpy`, and `matplotlib`. I avoided external dependencies beyond the standard scientific stack.

4. Results

4.1. Summary Statistics

Metric	Value
Annualised Return (simple)	10.09%
Annualised Volatility (simple)	13.81%
Sharpe Ratio (simple, 0% RFR)	0.73
Maximum Drawdown	-17.23%

4.2. Log-Return Equivalent

Metric	Value
Annualised Return (log)	9.04%
Annualised Volatility (log)	13.83%
Sharpe Ratio (log)	0.65

The log-return Sharpe is slightly lower because log returns are geometrically consistent.

4.3. Distribution of Daily Returns

Statistic	Value
Mean daily return	0.0381%

Statistic	Value
Standard deviation	0.8701%
Minimum daily return	-4.99%
Maximum daily return	+4.21%

The distribution is roughly symmetric, with a slightly heavier left tail — a common feature in equity markets.

4.4. Rolling One-Year Sharpe

Year	Rolling Sharpe
2022	0.55
2023	0.82
2024	0.79
2025	0.68

The Sharpe ratio is not constant. It varies from 0.55 to 0.82 across different periods. This suggests that market conditions matter — one static number isn't enough.

5. Discussion

5.1. Interpretation

A Sharpe ratio of 0.73 is average for a passive index. It means that for every unit of risk (13.81% volatility), the market delivered 0.73 units of excess return.

This is not exceptional, but it's also not bad. It's consistent with historical averages for large-cap equity indices.

5.2. What This Means for My Trading

The passive index gives me a baseline. To generate alpha, I need strategies that beat this number. My target for Phase 2 is a Sharpe > 1.0 for any active signal.

5.3. Limitations

- **Data source:** yfinance is convenient but not institutional-grade. I'll migrate to NSE bhavcopy later.
- **No transaction costs:** These results ignore brokerage, slippage, and STT — all of which reduce real-world Sharpe.
- **Single index:** The toolkit currently works only for Nifty 50. I'll extend it to 100+ stocks in Phase 2.

6. Manual Verification

I didn't trust my code. So I picked the first 10 days of data (after the NaN row) and calculated the Sharpe by hand.

- I wrote down the daily returns.
- I computed the mean.
- I computed the standard deviation (using sample std).
- I annualized both.
- I divided.

My hand calculation: 0.7312

My code: 0.7303

Difference: 0.0009

That's within 0.001. I trusted the code after that.

7. What I Learned

- **yfinance is finicky:** Always check column names. The nested multi-index caught me off guard.
 - **Manual verification is boring but necessary:** Without it, I would have assumed the code was correct and moved on. That's a dangerous habit.
 - **The Sharpe ratio is not static:** It moves with market conditions. Rolling windows are essential.
 - **A passive index gives 0.73 Sharpe:** To generate alpha, I need strategies that beat that.
-

8. Conclusion

I built a verifiable Python toolkit that downloads Nifty 50 data, computes log and simple returns, annualises them, and calculates Sharpe ratio and maximum drawdown. The pipeline passes manual verification and produces results consistent with known Indian market behaviour.

Next steps:

- Extend to 100+ stocks.
- Add transaction costs.
- Build alpha signals (momentum, mean reversion, carry).
- Event-driven backtesting.

The code is public. The methodology is documented. The results are reproducible.

9. Code Availability

All code for this paper is available at:

<https://github.com/shadow17ai-quants/Shadow-Oak-summer-sprint-2026>

Relevant files:

- [projects/niftyvault/downloader.py](#)
 - [projects/niftyvault/analyzer.py](#)
 - [projects/niftyvault/visualize.py](#)
-

References

- [yfinance documentation](#)
- [Mosh Hamedani, Python for Beginners](#)
- [Patrick Boyle, Sharpe Ratio Explained](#)
- [Corey Schafer, Pandas Tutorial](#)